Đại học Quốc gia Thành phố Hồ Chí Minh
Trường Đại học Bách khoa
Khoa Khoa học và Kĩ thuật Máy tính



THỰC HÀNH KIẾN TRÚC MÁY TÍNH
(CO2008)

Báo cáo Lab 2

Branches and procedures
(Lệnh rẽ nhánh và thủ tục)

Giảng viên hướng dẫn: Tôn Huỳnh Long
Nhóm: L01
Sinh viên: Đặng Minh Triết - 2510249

Thành phố Hồ Chí Minh, Tháng 11/2026

Mục lục

1 Bài 1 – Làm tròn về bội của 4 gần nhất **2**
1.1 Đề bài 2
1.2 Ý tưởng 2
1.3 Mã nguồn 3
1.4 Kết quả chạy trên MARS 6

Tài liệu tham khảo **7**

1 Bài 1 – Làm tròn về bội của 4 gần nhất

File code (Github):
Video demo (Github):
Video demo (Google Drive):

1.1 Đề bài

Viết một chương trình MIPS với các yêu cầu sau:

1. Khai báo một mảng số nguyên có độ dài 5 với dữ liệu được nhập từ bàn phím.
2. Kiểm tra từng phần tử trong mảng có chia hết cho 4 hay không.
3. Nếu một phần tử không chia hết cho 4, đổi nó thành số chia hết cho 4 gần nó nhất (ưu tiên làm tròn xuống). Ví dụ, nếu số là 35 thì kết quả là 36, nếu số là 34 hoặc 33 thì kết quả là 32.
4. In kết quả ra màn hình.

1.2 Ý tưởng

1. **Nhập mảng** (nhãn `input`): dành mảng `array` 20 byte bằng `.space 20` (5 phần tử, mỗi phần tử 4 byte). `$t0` là biến đếm, `$t1` là con trỏ tới phần tử đang nhập. Mỗi vòng lặp in lời nhắc `"Please input element <i>: "`, đọc số bằng `syscall` 5, lưu vào mảng bằng `sw $v0, 0($t1)`, tăng biến đếm thêm 1 và con trỏ thêm 4 byte. Khi đủ 5 phần tử (`beq $t0, 5, print_oldarray`) thì chuyển sang in mảng ban đầu.
2. **In mảng bằng thủ tục `printarray`:** mảng ban đầu và mảng sau khi làm tròn được in theo cùng một dạng `<lời nhắc>[a0, a1, a2, a3, a4]`, chỉ khác lời nhắc (`"Old array: "` hoặc `"New array: "`). Vì vậy phần in được viết thành một thủ tục dùng chung:
   - Nơi gọi (`print_oldarray`, `print_newarray`) đặt địa chỉ chuỗi lời nhắc vào `$a1` rồi gọi `jal printarray`. Lệnh `jal` lưu địa chỉ lệnh kế tiếp vào `$ra` rồi nhảy tới thủ tục.
   - Thủ tục in lời nhắc (`move $a0, $a1`) và `"["`, sau đó đặt `$t1` = địa chỉ `array[0]`, `$t0` = `array` +16 là địa chỉ phần tử cuối `array[4]`.
   - Vòng lặp `loop_printarray` in phần tử tại `$t1`. Nếu `$t1` = `$t0` (vừa in phần tử cuối) thì thoát vòng lặp; nếu chưa thì in `", "` rồi tăng `$t1` thêm 4. Nhờ kiểm tra điều kiện dừng **trước** khi in dấu phẩy nên không bị thừa dấu phẩy ở cuối.
   - Cuối thủ tục (`end_printarray`) in `"]"` và xuống dòng, rồi trở về nơi gọi bằng `jr $ra`. Sau khi in mảng cũ, chương trình nhảy sang `check`; sau khi in mảng mới thì kết thúc.
   - `printarray` là thủ tục lá (không gọi thủ tục nào khác) nên `$ra` không bị ghi đè, không cần cất `$ra` lên stack. Thủ tục có dùng `$t0`, `$t1`, nhưng đây là các thanh ghi tạm và `check` gán lại chúng ngay từ đầu nên không ảnh hưởng.
3. **Kiểm tra chia hết cho 4** (nhãn `check`/`div_4`): đặt `$s0` = 4, với mỗi phần tử x dùng `div $t2, $s0`, lấy **thương** q bằng `mflo $t3` và **số dư** r bằng `mfhi $t4`. Nếu $r = 0$ thì x chia hết cho 4, giữ nguyên và xét phần tử kế tiếp; nếu $r \neq 0$ thì nhảy tới `change`.
4. **Đổi thành bội của 4 gần nhất** (nhãn `change`): kết quả luôn có dạng $k \times 4$, chỉ cần chọn đúng k. Lệnh `div` của MIPS chia làm tròn về 0, nên số dư **cùng dấu với số bị chia**: $x = 4q + r$ với $r \in \{1, 2, 3\}$ khi $x > 0$ và $r \in \{-1, -2, -3\}$ khi $x < 0$.

- $x > 0$**:** hai bội gần nhất là $4q$ (nhỏ hơn x, cách r) và $4(q+1)$ (lớn hơn x, cách $4-r$).
  - $r = 1$: $4q$ gần hơn $\Rightarrow k = q$.
  - $r = 2$: cách đều, ưu tiên làm tròn xuống $\Rightarrow k = q$.
  - $r = 3$: $4(q+1)$ gần hơn $\Rightarrow k = q+1$ (nhãn `round_up_positive`).
- $x < 0$**:** thương bị làm tròn về 0 nên $4q$ lại là bội **lớn hơn** x (cách $|r|$), còn bội nhỏ hơn là $4(q-1)$ (cách $4-|r|$).
  - $r = -1$: $4q$ gần hơn $\Rightarrow k = q$.
  - $r = -2$: cách đều, ưu tiên làm tròn xuống (về phía giá trị nhỏ hơn) $\Rightarrow k = q-1$.
  - $r = -3$: $4(q-1)$ gần hơn $\Rightarrow k = q-1$.

Trong code, các trường hợp $r = 1, 2, -1$ nhảy thẳng tới `save`; $r = 3$ nhảy tới `round_up_positive` để tăng `$t3` thêm 1; hai trường hợp còn lại ($r = -2, -3$) giảm `$t3` đi 1. Tại `save`, tính `$t3` $= k \times 4$ bằng `mul`, ghi đè lại vào mảng bằng `sw $t3, 0($t1)` rồi quay lại `div_4` xét phần tử kế tiếp (hoặc sang in mảng mới nếu đã xét xong phần tử cuối).

5. Các lệnh `beq $t4, 1, save`, `bne $t4, 0, change` và `mul $t3, $t3, 4` có toán hạng là hằng số nên là lệnh giả: MARS tự nạp hằng số vào thanh ghi `$at` trước rồi mới thực hiện lệnh với `$at`.

Bảng 1: Các trường hợp làm tròn (thương q từ `mflo`, số dư r từ `mfhi`)

x	q	r	Hai bội gần nhất	k	Kết quả $4k$
35	8	3	32, **36**	$q+1=9$	36
34	8	2	**32**, 36	$q=8$	32
33	8	1	**32**, 36	$q=8$	32
32	8	0	–	–	32 (giữ nguyên)
-33	-8	-1	-36, $-$**32**	$q=-8$	-32
-34	-8	-2	$-$**36**, -32	$q-1=-9$	-36
-35	-8	-3	$-$**36**, -32	$q-1=-9$	-36

1.3 Mã nguồn

```
.data
    array: .space 20
    prompt_input: .asciiz "Please input element "
    colon: .asciiz ": "
    prompt_oldarray: .asciiz "Old array: "
    prompt_newarray: .asciiz "New array: "
    square_bracket_begin: .asciiz "["
    square_bracket_end: .asciiz "]"
    comma: .asciiz ", "
    newline: .asciiz "\n"

.text
.globl main
main:
    li $t0, 0 # Bien dem
```

```
16      la $t1, array # array chua 5 so nguyen
17
18  # 1. Input
19  input: # Nhap 5 so nguyen
20      # In prompt
21      li $v0, 4
22      la $a0, prompt_input
23      syscall
24
25      li $v0, 1
26      move $a0, $t0
27      syscall
28
29      li $v0, 4
30      la $a0, colon
31      syscall
32
33      # Nhap so
34      li $v0, 5
35      syscall
36
37      # Cap nhat bien dem va array
38      sw $v0, 0($t1)
39      addi $t0, $t0, 1
40      addi $t1, $t1, 4
41      beq $t0, 5, print_oldarray
42      j input
43
44  print_oldarray: # In array cu
45      la $a1, prompt_oldarray
46      jal printarray
47      j check # Chuyen qua check so co chia het cho 4 ko
48
49  printarray:
50      li $v0, 4
51      move $a0, $a1
52      syscall
53
54      li $v0, 4
55      la $a0, square_bracket_begin
56      syscall
57
58      la $t1, array # dia chi array[0]
59      la $t0, array
60      addi $t0, $t0, 16 # dia chi array[4]
61
62  loop_printarray:
63      li $v0, 1
64      lw $a0, 0($t1)
65      syscall
66
67      beq $t1, $t0, end_printarray
68
69      li $v0, 4
70      la $a0, comma
71      syscall
72
73      addi $t1, $t1, 4
74      j loop_printarray
75
76
```

```
77  end_printarray:
78      li $v0, 4
79      la $a0, square_bracket_end
80      syscall
81
82      li $v0, 4
83      la $a0, newline
84      syscall
85      jr $ra # Ve lai print_(old/new)array
86
87  # 2. Check
88  check: # Kiem tra chia het cho 4
89      la $t1, array # dia chi array[0]
90      la $t0, array
91      addi $t0, $t0, 16 # dia chi array[4]
92      li $s0, 4
93
94  div_4:
95      lw $t2, 0($t1)
96      div $t2, $s0
97      mflo $t3 # Thuong
98      mfhi $t4 # So du
99      bne $t4, 0, change # Chuyen qua change so ko chia het cho 4
100     beq $t1, $t0, print_newarray # Chuyen qua in array moi
101     addi $t1, $t1, 4
102     j div_4
103
104 change: # Chinh so ko chia het cho 4
105     # *Xet so duong (so > 0)
106     # - Neu du 1 -> thuong * 4 (round down)
107     # - Neu du 2 -> thuong * 4 (round down)
108     # - Neu du 3 -> (thuong + 1) * 4
109     #  *Xet so am (so < 0)
110     # - Neu du -1 -> thuong * 4 (round down)
111     # - Neu du -2 -> (thuong - 1) * 4
112     # - Neu du -3 -> (thuong - 1) * 4
113     beq $t4, 1, save
114     beq $t4, 2, save
115     beq $t4, 3, round_up_positive
116     beq $t4, -1, save
117     addi $t3, $t3, -1 # Lam tron xuong
118     j save
119
120 round_up_positive: # Lam tron len
121     addi $t3, $t3, 1
122     j save
123
124 save: # Luu lai so moi
125     mul $t3, $t3, 4
126     sw $t3, 0($t1)
127     beq $t1, $t0, print_newarray # Chuyen qua in array moi
128     addi $t1, $t1, 4
129     j div_4
130
131 print_newarray: # In array moi
132     la $a1, prompt_newarray
133     jal printarray
134     j end # Ket thuc chuong trinh
135
136 end:
137     li $v0, 10
```

```
138     syscall
```

1.4 Kết quả chạy trên MARS

Chạy với các số trong ví dụ của đề (35, 34, 33) cùng hai số 32 và 10:

```
Please input element 0: 35
Please input element 1: 34
Please input element 2: 33
Please input element 3: 32
Please input element 4: 10
Old array: [35, 34, 33, 32, 10]
New array: [36, 32, 32, 32, 8]
```

Chạy với các số âm và số 0:

```
Please input element 0: -33
Please input element 1: -34
Please input element 2: -35
Please input element 3: -36
Please input element 4: 0
Old array: [-33, -34, -35, -36, 0]
New array: [-32, -36, -36, -36, 0]
```

Tài liệu tham khảo

[1] Pete Sanderson, Kenneth Vollmar. *MARS (MIPS Assembler and Runtime Simulator), phiên bản 4.5.* Truy cập vào 02/11/2026. https://dpetersanderson.github.io/

[2] David A. Patterson, John L. Hennessy. *Computer Organization and Design: The Hardware/Software Interface, MIPS Edition.* Morgan Kaufmann.

[3] Khoa Khoa học và Kĩ thuật Máy tính, Trường Đại học Bách khoa – ĐHQG-HCM. *Computer Architecture Lab (CO2008) – Lab 2: Branches and procedures.* 2025.